Onyx — GPIOKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **GPIOKit** (`SD:/lib/gpiokit.so`, `user/Kits/gpiokit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `gpiokit/gpiokit.h`

What it is

GPIOKit is the Raspberry Pi's 40-pin header: a pin's mode and level, PWM, a servo, edges queued with their time, the I2C bus and SPI, the header's own table — and a simulated board (an SSD1306 display and a BME280 sensor on its I2C bus) for a PC or a system without the hardware. The levels are 3.3 V.

Include	`#include "gpiokit/gpiokit.h"`
Link	`lib/gpiokit.imp.a` (C++) or `lib/gpiokit.imp_c.a` (C)
Library	`SD:/lib/gpiokit.so` — 34 entries in its table (`user/Kits/gpiokit/gpiokit.abi`, append-only)
Sources	`user/Kits/gpiokit/`

Using it

The Raspberry Pi's header from a program: a pin's mode, its level, PWM, edges, the I2C bus and SPI. Pins are **BCM GPIO numbers** (GPIO 17 is the header's pin 11: `gk_header_pin`). **The levels are 3.3 V**: never wire 5 V to a pin; at most about 16 mA a pin (an LED through a 330 Ω resistor). A pin is the program's from its first use until it gives it back (`gk_mode (pin, GK_FREE)`, `gk_release`) or ends — the system then makes it an input again, after a crash too. The system's pins (GPIO 14 / 15, the serial console; 0 / 1, the HAT EEPROM) and a pin another program has are refused: every call returns ≥ 0, or a negative `GK_E*` that `gk_error` says in words.

GPIOKit is the one kit besides AppKit that calls the kernel itself (kapi v92 `gpio_ctl`; the user's exception, docs/03 §5.10); it is shipped with the kernel in the package `onyx`.

An LED, a button (C):

```
#include "gpiokit/gpiokit.h"

gk_mode (17, GK_OUT);                                  // the LED: GPIO 17 -> 330 ohm -> LED
```

```
-> GND
gk_mode (27, GK_IN_PULLUP);                          // the button: GPIO 27 -> button -> GND
(reads 0 pressed)
gk_edges (27, GK_FALLING);                           // its presses queued, with their time
for (;;)
{
    gk_event e[8];
    int n = gk_events (e, 8, 1000);                  // waits up to 1 s for the first
    for (int i = 0; i < n; i++) gk_toggle (17);      // each press turns the LED over
}
```

PWM and a servo (GPIO 12, 13, 18, 19; 12 / 18 and 13 / 19 share a channel):

```
gk_pwm (18, 1000, 2500);                             // 1 kHz, 25 % high (duty in 1/10000)
gk_servo (18, 1500);                                 // a servo: 50 Hz, a 1.5 ms pulse (the
middle)
```

I2C (GPIO 2 SDA, GPIO 3 SCL) — a BME280's id, a scan:

```
if (gk_i2c_open (0) == 0)                            // 0: 100 kHz
{
    unsigned char map[16];
    int n = gk_i2c_scan (map);                       // bit a of map: something answers at a
    if (gk_i2c_reg_read (0x76, 0xD0) == 0x60) ax_putln ("a BME280 at 0x76");
    gk_i2c_close ();
}
```

SPI 0 (GPIO 8 CE0, 7 CE1, 9 MISO, 10 MOSI, 11 SCLK): `gk_spi_open (1000000, 0)`, then `gk_spi_transfer (0, tx, rx, n)`.

C++ has small classes that give the pin back by themselves:

```
gpiokit::Pin led (17, GK_OUT);
if (!led.ok ()) ax_putln (gk_error (led.status ()));
led.write (1);
gpiokit::I2CDevice bme (0x76);
int id = bme.reg (0xD0);
```

No hardware? GPIOKit has a **simulator** — a board in memory: the inputs set by `gk_sim_input`, the PWM's waves, an I2C bus with an SSD1306 display (0x3C: `gk_sim_display` gives its pixels) and a BME280 sensor (0x76), SPI looped back. `gk_sim (1)` switches to it; it runs by itself where the system has no GPIO, and on a PC. GPIO Lab (docs/04) is built on GPIOKit; BASIC has statements for it (docs/04 §13 *GPIO*).

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`gk_pin_info`	A pin as the system sees it (gk_info)	`gpiokit.h`
`gk_event`	An edge (gk_events)	`gpiokit.h`
`gk_available`	What drives the pins	`gpiokit.h`
`gk_sim`	The simulator on (1	`gpiokit.h`
`gk_error`	A negative code's text ("the system uses this	`gpiokit.h`

Name	What it does	Header
	pin").	
`gk_release`	Every pin, bus and edge of the program's given back (its end does it too) -> 0.	`gpiokit.h`
`gk_now_us`	The clock of the edges' times, now (microseconds	`gpiokit.h`
`gk_mode`	The pin set to a mode (GK_FREE ...	`gpiokit.h`
`gk_write`	An output of the program's set high (1) or low (0) -> 0 / GK_E*.	`gpiokit.h`
`gk_toggle`	An output turned over -> its new level / GK_E*.	`gpiokit.h`
`gk_read`	A pin's level now -> 0 / 1 / GK_EINVAL (any pin of the header may be read, whoever has it).	`gpiokit.h`
`gk_read_all`	Every level at once	`gpiokit.h`
`gk_info`	The pins as the system sees them, GPIO 0 first, up to max (GK_PINS	`gpiokit.h`
`gk_mode_name`	A mode's name ("input", "pull-up", "output", "PWM", "I2C", "ALT3"...).	`gpiokit.h`
`gk_pwm`	GPIO 12 / 13 / 18 / 19 made a square wave of freq_hz (1 ..	`gpiokit.h`
`gk_servo`	A hobby servo on a PWM pin	`gpiokit.h`
`gk_edges`	An input of the program's	`gpiokit.h`
`gk_events`	The edges that came, up to max, waiting up to wait_ms (0 ..	`gpiokit.h`
`gk_i2c_open`	The I2C bus 1 (GPIO 2 SDA, GPIO 3 SCL	`gpiokit.h`
`gk_i2c_close`	The bus given back -> 0.	`gpiokit.h`
`gk_i2c_write`	n bytes written to the device at addr (7 bits	`gpiokit.h`
`gk_i2c_read`	n bytes read from the device -> n / GK_EIO.	`gpiokit.h`

Name	What it does	Header
`gk_i2c_write_read`	wn bytes written, then rn read (a repeated start between	`gpiokit.h`
`gk_i2c_reg_read`	A register of 8 bits read -> its value (0..255) / GK_E*.	`gpiokit.h`
`gk_i2c_reg_write`	A register of 8 bits written -> 0 / GK_E*.	`gpiokit.h`
`gk_i2c_scan`	Every address 0x08..0x77 tried	`gpiokit.h`
`gk_i2c_guess`	What usually answers at an I2C address ("SSD1306 display", "BME280 / BMP280 sensor"...), or "".	`gpiokit.h`
`gk_spi_open`	SPI 0 the program's (GPIO 8 CE0, 7 CE1, 9 MISO, 10 MOSI, 11 SCLK), at clock_hz (0	`gpiokit.h`
`gk_spi_transfer`	n bytes sent while n come back, chip select cs (0	`gpiokit.h`
`gk_spi_close`	SPI 0 given back -> 0.	`gpiokit.h`
`gk_header_gpio`	The 40 pins as printed on the board, pin 1 (3V3, top left with the board's USB ports down) to 40.	`gpiokit.h`
`gk_header_pin`	A GPIO's header pin -> 1..40, or -1.	`gpiokit.h`
`gk_header_label`	A header pin's label	`gpiokit.h`
`gk_gpio_function`	What else a GPIO is, if anything	`gpiokit.h`
`gk_sim_input`	(gk_sim (1) first, or a PC build.) What drives a simulated input from outside	`gpiokit.h`
`gk_sim_display`	What the simulated SSD1306 shows	`gpiokit.h`
`Pin`	A pin of the program's	`gpiokit.h`
`Pwm`	A PWM output (GPIO 12, 13, 18 or 19).	`gpiokit.h`
`I2CDevice`	A device on the I2C bus (the bus opened by the first one made, if it was	`gpiokit.h`

Name	What it does	Header
	not).	

gpiokit/gpiokit.h

gpiokit.h -- GPIOKit, the Raspberry Pi's 40-pin header for the programs (SD:/lib/gpiokit.so; docs/06-KITS-GUIDE.md "GPIOKit", docs/19-GPIOKIT.md).

```
  pins       a pin's mode (input, pulled up / down, output, an alternate function), its level
             written and read, every level at once (gk_mode, gk_write, gk_read, gk_read_all)
  PWM        a square wave on GPIO 12 / 13 / 18 / 19: its frequency and its duty; a servo's
pulse
             (gk_pwm, gk_servo)
  edges      a pin's rising / falling edges queued, with their time, and taken (gk_edges,
gk_events)
  I2C        the bus on GPIO 2 (SDA) / 3 (SCL): transfers, a device's registers, a scan
(gk_i2c_*)
  SPI        SPI 0 on GPIO 7..11: full-duplex transfers (gk_spi_*)
  the header the 40 pins as printed on the board: their numbers, names, functions
(gk_header_*)
  simulator  a board in memory -- inputs set by hand, the PWM's waves, an I2C bus with an
SSD1306
             display and a BME280 sensor, SPI looped back -- for a PC, or an Onyx without the
             hardware (gk_sim, gk_sim_input)
```

Pins are BCM GPIO numbers (GPIO 17 is the header's pin 11). The levels are 3.3 V: never wire 5 V to a pin. A pin is the program's from its first use until gk_mode (pin, GK_FREE), gk_release or the program's end -- then the system makes it an input again. Pins the system uses (the serial console's GPIO 14 / 15, the HAT EEPROM's 0 / 1) are refused, and so is a pin another program has.

Every call that can fail returns a negative GK_E* code (gk_error says it in words); >= 0 is success. A C++ program also has small classes over these (namespace gpiokit: Pin, Pwm, I2CDevice).

Link lib/gpiokit.imp.a (C++) or lib/gpiokit.imp_c.a (C). The interface is append-only (gpiokit.abi). GPIOKit is the one kit besides AppKit that calls the kernel's table itself (kapi gpio_ctl, v92; the user's exception, docs/03 §5.10): it is shipped and rebuilt with the kernel, in the package onyx.

MIT License

Copyright (c) 2026 Stéphane Wegener and the Onyx contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF

CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

```
#define GK_PINS      28      // GPIO 0..27: the header's
```

modes, edges, errors

(the same numbers as the kernel's KAPI_GPIO_M_*)

```
#define GK_FREE      0       // nobody's: an input, no pull (gk_mode gives the pin back)
#define GK_IN        1       // an input, floating
#define GK_IN_PULLUP     2       // an input pulled up (a button to ground reads 0 when
pressed)
#define GK_IN_PULLDOWN   3       // an input pulled down
#define GK_OUT       4       // an output (starts low)
#define GK_PWM       5       // a PWM output (gk_pwm sets it)
#define GK_I2C       6       // a pin of the I2C bus (gk_i2c_open)
#define GK_SPI       7       // a pin of SPI 0 (gk_spi_open)
#define GK_ALT0      8       // GK_ALT0 + n: alternate function n (0..5) chosen by hand

#define GK_RISING    1       // gk_edges: the edges wanted
#define GK_FALLING   2
#define GK_BOTH      3

#define GK_EPERM     (-1)        // the system's pin, or not the program's
#define GK_EIO       (-5)        // no answer on the bus
#define GK_ENOMEM    (-12)
#define GK_EFAULT    (-14)
#define GK_EBUSY     (-16)       // another program has it
#define GK_ENODEV    (-19)       // no GPIO here (a system older than kapi 92): gk_sim (1)
simulates
#define GK_EINVAL    (-22)       // a wrong pin, mode, frequency, duty...
```

gk_pin_info.flags

```
#define GK_F_RESERVED    1       // the system's: never given (reason says whose)
#define GK_F_PWM     2       // can do PWM (12, 13, 18, 19)
#define GK_F_EDGES   4       // its edges are queued for its owner
```

A pin as the system sees it (gk_info): the same layout as the kernel's struct kapi_gpio_pin.

```
typedef struct gk_pin_info
{
    unsigned char pin;       // GPIO number
    unsigned char mode;      // GK_FREE ... GK_ALT0 + 5
    unsigned char level;         // 0 / 1 now
    unsigned char flags;         // GK_F_*
    unsigned      owner;         // the program (its pid) that has it, 0: none
    unsigned      pwm_freq;      // Hz, in PWM
    unsigned      pwm_duty;      // 1/10000, in PWM
    char          reason[24];    // why it is reserved ("serial console")
} gk_pin_info;
```

An edge (gk_events): the same layout as the kernel's struct kapi_gpio_event.

```
typedef struct gk_event
{
    unsigned char pin;
    unsigned char edge;      // GK_RISING / GK_FALLING
```

```
    unsigned char level;        // the level just after it
    unsigned char lost;     // 1: edges were dropped before this one (the queue was full)
    unsigned      reserved;
    unsigned long long us;      // when, in microseconds (gk_now_us's clock)
} gk_event;
```

the kit

What drives the pins: 1 the Pi's header, 2 the simulator, 0 nothing (gk_sim (1) gives the simulator).

```
int gk_available (void);
```

The simulator on (1: every call after goes to a board in memory) or off (0: the real header, if there is one) -> what drives the pins now (as gk_available). A PC build of GPIOKit is always simulated.

```
int gk_sim (int on);
```

A negative code's text ("the system uses this pin").

```
const char *gk_error (int code);
```

Every pin, bus and edge of the program's given back (its end does it too) -> 0.

```
int gk_release (void);
```

The clock of the edges' times, now (microseconds; it only goes forward).

```
unsigned long long gk_now_us (void);
```

pins

The pin set to a mode (GK_FREE ... GK_OUT, GK_ALT0 + n) -> 0 / GK_E*.

```
int gk_mode (int pin, int mode);
```

An output of the program's set high (1) or low (0) -> 0 / GK_E*.

```
int gk_write (int pin, int level);
```

An output turned over -> its new level / GK_E*.

```
int gk_toggle (int pin);
```

A pin's level now -> 0 / 1 / GK_EINVAL (any pin of the header may be read, whoever has it).

```
int gk_read (int pin);
```

Every level at once: bit n = GPIO n.

```
unsigned gk_read_all (void);
```

The pins as the system sees them, GPIO 0 first, up to max (GK_PINS: all) -> how many / GK_E*.

```
int gk_info (gk_pin_info *out, int max);
```

A mode's name ("input", "pull-up", "output", "PWM", "I2C", "ALT3"...).

```
const char *gk_mode_name (int mode);
```

PWM

GPIO 12 / 13 / 18 / 19 made a square wave of freq_hz (1 .. 1 000 000) high for duty / 10000 of the time (0 .. 10000: 2500 = 25 %) -> 0 / GK_E*. 12 and 18 share a channel, 13 and 19 the other: one of each pair. The pin stays in PWM until gk_mode gives it another mode.

```
int gk_pwm (int pin, int freq_hz, int duty);
```

A hobby servo on a PWM pin: 50 Hz, a pulse of pulse_us (500 .. 2500; 1500: the middle) -> 0 / GK_E*.

```
int gk_servo (int pin, int pulse_us);
```

edges

An input of the program's: its edges (GK_RISING | GK_FALLING; 0: none) queued for it -> 0 / GK_E*. The queue holds 256 edges; a button bounces: a few edges a press.

```
int gk_edges (int pin, int edges);
```

The edges that came, up to max, waiting up to wait_ms (0 .. 1000) for the first -> how many / GK_E*.

```
int gk_events (gk_event *out, int max, int wait_ms);
```

I2C

The I2C bus 1 (GPIO 2 SDA, GPIO 3 SCL; the board has its pull-ups) the program's, at clock_hz (0: 100 kHz; 400000 for fast devices) -> 0 / GK_E*.

```
int gk_i2c_open (int clock_hz);
```

The bus given back -> 0.

```
int gk_i2c_close (void);
```

n bytes written to the device at addr (7 bits: 0x3C) -> n / GK_EIO (no answer).

```
int gk_i2c_write (int addr, const void *data, int n);
```

n bytes read from the device -> n / GK_EIO.

```
int gk_i2c_read (int addr, void *data, int n);
```

wn bytes written, then rn read (a repeated start between: a register's address, then its bytes) -> rn / GK_E*.

```
int gk_i2c_write_read (int addr, const void *wr, int wn, void *rd, int rn);
```

A register of 8 bits read -> its value (0..255) / GK_E*.

```
int gk_i2c_reg_read (int addr, int reg);
```

A register of 8 bits written -> 0 / GK_E*.

```
int gk_i2c_reg_write (int addr, int reg, int value);
```

Every address 0x08..0x77 tried: bit a of map (16 bytes: map[a / 8] >> (a % 8)) set where a device answered -> how many answered / GK_E*.

```
int gk_i2c_scan (unsigned char map[16]);
```

What usually answers at an I2C address ("SSD1306 display", "BME280 / BMP280 sensor"...), or "".

```
const char *gk_i2c_guess (int addr);
```

SPI

SPI 0 the program's (GPIO 8 CE0, 7 CE1, 9 MISO, 10 MOSI, 11 SCLK), at clock_hz (0: 1 MHz), mode 0..3 (CPOL << 1 | CPHA) -> 0 / GK_E*. Again: the clock and the mode changed.

```
int gk_spi_open (int clock_hz, int mode);
```

n bytes sent while n come back, chip select cs (0: CE0, 1: CE1); tx 0: zeros sent, rx 0: dropped -> n / GK_E*.

```
int gk_spi_transfer (int cs, const void *tx, void *rx, int n);
```

SPI 0 given back -> 0.

```
int gk_spi_close (void);
```

the header

The 40 pins as printed on the board, pin 1 (3V3, top left with the board's USB ports down) to 40. A header pin's GPIO -> 0..27, or -1 (a power or ground pin).

```
int gk_header_gpio (int header_pin);
```

A GPIO's header pin -> 1..40, or -1.

```
int gk_header_pin (int gpio);
```

A header pin's label: "3V3", "5V", "GND", "GPIO17"...

```
const char *gk_header_label (int header_pin);
```

What else a GPIO is, if anything: "SDA1", "SCL1", "TXD0", "PWM0", "SPI0 CE0", "ID_SD"... or "".

```
const char *gk_gpio_function (int gpio);
```

the simulator

(gk_sim (1) first, or a PC build.) What drives a simulated input from outside: level 0 / 1, or -1 (nothing: the pin floats, or follows its pull) -> 0. An edge is queued if the program asked for it.

```
int gk_sim_input (int pin, int level);
```

What the simulated SSD1306 shows: its 128 x 64 pixels, a byte for 8 rows (its own memory layout: page p, column c at p * 128 + c) -> out (1024 bytes) filled, 1 / 0 (the simulator is off).

```
int gk_sim_display (unsigned char out[1024]);

}
```

A pin of the program's: its mode set at its making, given back at its end.

```
class Pin
{
public:
```

```
    Pin (int gpio, int mode) : gpio_ (gpio) { status_ = gk_mode (gpio, mode); }
    ~Pin () { if (status_ == 0) gk_mode (gpio_, GK_FREE); }
    Pin (const Pin &) = delete;
    Pin &operator= (const Pin &) = delete;
    bool ok () const { return status_ == 0; }
    int status () const { return status_; }     // 0, or why it failed (gk_error)
    int gpio () const { return gpio_; }
    int read () const { return gk_read (gpio_); }
    int write (int level) const { return gk_write (gpio_, level); }
    int toggle () const { return gk_toggle (gpio_); }
    int edges (int which) const { return gk_edges (gpio_, which); }
private:
    int gpio_, status_;
};
```

A PWM output (GPIO 12, 13, 18 or 19).

```
class Pwm
{
public:
    explicit Pwm (int gpio, int freq_hz = 1000, int duty = 0) : gpio_ (gpio), freq_ (freq_hz)
{ status_ = gk_pwm (gpio, freq_hz, duty); }
    ~Pwm () { if (status_ == 0) gk_mode (gpio_, GK_FREE); }
    Pwm (const Pwm &) = delete;
    Pwm &operator= (const Pwm &) = delete;
    bool ok () const { return status_ == 0; }
    int duty (int d) { return gk_pwm (gpio_, freq_, d); }       // 0 .. 10000
    int frequency (int hz, int d) { freq_ = hz; return gk_pwm (gpio_, hz, d); }
    int servo (int pulse_us) { freq_ = 50; return gk_servo (gpio_, pulse_us); }
private:
    int gpio_, freq_, status_;
};
```

A device on the I2C bus (the bus opened by the first one made, if it was not).

```
class I2CDevice
{
public:
    explicit I2CDevice (int addr, int clock_hz = 0) : addr_ (addr) { status_ = gk_i2c_open
(clock_hz); }
    bool ok () const { return status_ == 0; }
    int addr () const { return addr_; }
    int write (const void *d, int n) const { return gk_i2c_write (addr_, d, n); }
    int read (void *d, int n) const { return gk_i2c_read (addr_, d, n); }
    int write_read (const void *w, int wn, void *r, int rn) const { return gk_i2c_write_read
(addr_, w, wn, r, rn); }
    int reg (int r) const { return gk_i2c_reg_read (addr_, r); }
    int reg (int r, int v) const { return gk_i2c_reg_write (addr_, r, v); }
private:
    int addr_, status_;
};
```